Relatório breve — Processo de commits

Projecto: Agenda de Investimentos

Grupo: Azael Emídio • Feidão Décimo • Elídio Arsénio • Margarete de Carvalho

O que fizemos

Trabalhámos com Git para guardar versões do projecto e publicá-lo no GitHub. Começámos por inicializar um repositório local com git init, preparar os ficheiros com git add e criar a branch main. Em seguida, configurámos o repositório remoto, fizemos a autenticação pelo navegador e executámos o envio com git push.

Para colocar o projecto dentro do repositório já existente FacilitaMais, adoptámos depois um procedimento mais seguro: clonámos o repositório, copiámos os ficheiros do projecto para a pasta agenda-investim e preparámos essa pasta para um novo commit, sem transportar a pasta .git interna, as dependências node_modules ou o ficheiro .env.

Desafios que encontrámos

- **Identificação do autor:** o primeiro commit foi recusado porque o Git ainda não tinha nome e e-mail configurados. Percebemos que esses dados identificam quem regista cada commit.
- **Repositório dentro de outro:** como o projecto já tinha a sua própria pasta .git, ao trabalhar a partir da pasta superior o Git detectou-o como repositório incorporado (referência de submódulo), em vez de adicionar os ficheiros individualmente.
- **Remoto e endereço:** tivemos erros ao definir o remoto por causa do nome do remoto e do endereço utilizado. Aprendemos a confirmar a configuração com git remote -v e a usar o endereço do repositório, não um link de navegação para uma subpasta.
- **Preparar não é publicar:** em determinado momento, git status mostrou “Changes to be committed”, mas ainda não existia um novo commit. Por isso, git push respondeu “Everything up-to-date”. Faltava executar git commit antes do envio.

O que aprendemos

Consolidámos a diferença entre três etapas: git add prepara as alterações; git commit guarda essas alterações no histórico local; e git push envia os commits para o repositório remoto. Também aprendemos a usar git status para verificar o estado dos ficheiros, git log para consultar o histórico e git remote -v para confirmar o destino do envio.

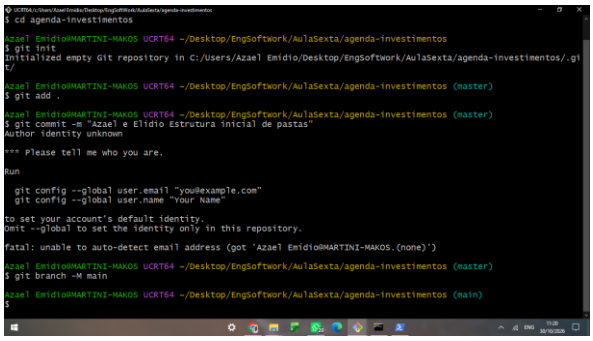
Percebemos ainda que os avisos LF/CRLF no Windows dizem respeito ao formato das quebras de linha e, por si só, não significam que o comando tenha falhado. Por fim, vimos que devemos evitar publicar credenciais e ficheiros desnecessários, como .env e node_modules.

Conclusão

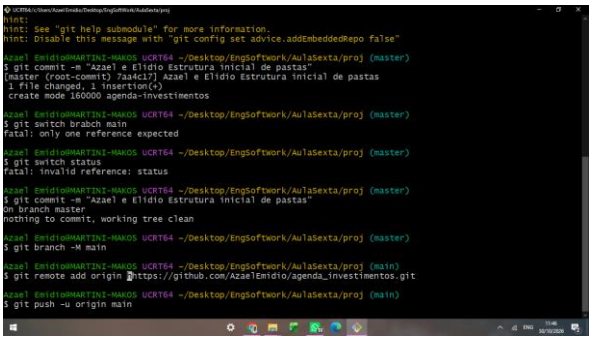
O processo exigiu algumas tentativas e correcções, mas ajudou-nos a compreender melhor o fluxo de trabalho com Git e GitHub. Aprendemos que não basta executar comandos em sequência: é importante confirmar a pasta em que estamos, verificar o estado do repositório e distinguir um commit local de um envio para o GitHub. Estas práticas tornam o trabalho em grupo mais organizado e facilitam o acompanhamento das alterações.

Registo visual do processo

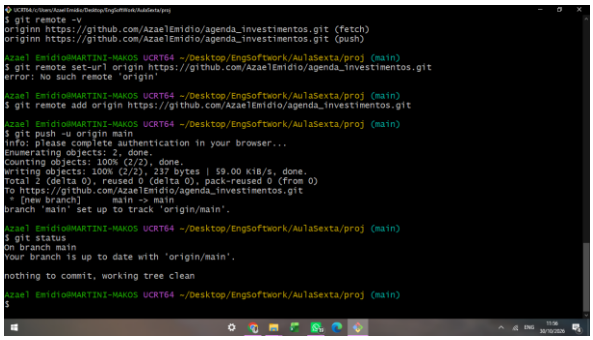
Seleccionámos quatro capturas que mostram alguns dos principais momentos e dificuldades.



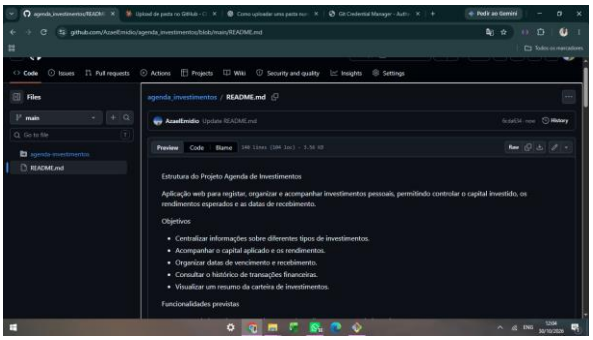
1. Primeiro commit recusado: faltava configurar a identidade do autor no Git.



2. O Git detectou um repositório dentro de outro e registou uma referência de submódulo.



3. Autenticação concluída e envio da branch main para o repositório remoto.



4. Visualização do repositório agenda_investimentos no GitHub.